24. Codex Token Saving

Frontmatter

video_number: 24
folder_title: Codex_Token_Saving
series: Building an AI Business From Scratch
video_type: Systems / Optimization
hook_type: Problem hook -- billing surprise
framework_phrase: Scope, model, monitor
script_versions: 1
status: ideation

Reference Videos

- How OpenAI's New Codex Pricing Turns Into a Billing Trap for Builders -- https://www.youtube.com/watch?v=IYxn4VyG0Fc
- How to Check Codex Token Limits & Account Usage -- https://www.youtube.com/watch?v=lVC1uw5PClk
- OpenAI Codex Explained for Normal People -- https://www.youtube.com/watch?v=ZkGzEMDeJos

The One Thing

Codex's per-task billing model rewards tight task scoping in a way Claude Code's per-token model doesn't -- understanding this difference changes how you work.

Hook

Codex runs on o3. Every task spins up reasoning. Here's how I cut my Codex bill by 60% without slowing down what I ship.

Target Viewer

Codex CLI users who got their first invoice and were surprised, or Claude Code users evaluating Codex who want to understand the real cost before switching.

Payoff

After watching, the viewer understands how Codex bills per task, knows the 4 levers that control cost, and can apply at least 2 of them immediately to reduce spend without changing output.

Beat Structure

Beat -- 1 -- How Codex Bills

Point: Codex charges per task completion -- it spins up o3, runs reasoning, takes actions, and bills for the full token footprint of that reasoning chain, not just input/output.

Visual: Diagram: task submitted -> o3 reasoning chain -> actions -> output -> bill

Target words: per-task, reasoning tokens, o3 cost model

Beat -- 2 -- What Drives Cost Up

Point: Vague tasks trigger long reasoning chains. File-heavy contexts balloon token counts. Multiple back-and-forth confirmations multiply the cost of a single logical task.

Visual: Split: vague task cost breakdown vs specific task cost breakdown

Target words: reasoning chain, vague task, confirmation loop

Beat -- 3 -- Task Scoping

Point: One well-scoped task with a clear deliverable costs a fraction of three vague tasks that converge on the same result -- scope is the highest-leverage cost lever.

Visual: Before/after: vague task prompt vs scoped task prompt, cost comparison

Target words: task scope, deliverable, prompt precision

Beat -- 4 -- File Exclusion and Model Selection

Point: Codex ingests files to build context -- using .codexignore to exclude irrelevant files and choosing a lighter model for simple tasks both cut cost significantly.

Visual: Terminal: .codexignore file contents, model selection flag in Codex command

Target words: .codexignore, model selection, context size

Beat -- 5 -- Monitor and Audit

Point: Codex usage dashboard shows cost per task -- reviewing it weekly identifies which task types are expensive and informs future scoping decisions.

Visual: Screen recording: Codex usage dashboard, per-task cost breakdown

Target words: usage dashboard, per-task audit, cost visibility

Analogy Bank

- Per-task billing vs per-token billing: like hiring a consultant (Codex -- you pay for the engagement, reasoning included) vs a copywriter who charges per word (Claude Code -- every token counts). [concept: billing model difference]
- Task scoping: like a precise surgery vs exploratory surgery -- exploratory costs 3x more and sometimes finds nothing new. [concept: why scope matters for cost]

Thesis Line Location

Beat 1 -- the core insight is that Codex bills for reasoning tokens you never see, which means vague tasks are expensive in a way that's invisible until you check the dashboard.

Framework Phrase

Scope, model, monitor

Specificity Bank

- o3 model -- Codex CLI default reasoning model
- .codexignore -- file exclusion mechanism in Codex CLI
- Codex usage dashboard -- in platform.openai.com under usage

- 60% cost reduction possible through task scoping alone (observed pattern)
- Reasoning tokens are billed at a higher rate than standard input tokens in o3

Constraints

- Do not give exact price figures -- they change; use proportional comparisons
- Do not recommend avoiding Codex -- frame as using it efficiently
- Focus on CLI usage, not the Codex web UI or API

CTA Direction

Next video: there's a new class of agents that go beyond both Claude Code and Codex -- they run 24/7, learn from every session, and don't need you to stay in the terminal.

Personal Angle

[DANIEL TO FILL] -- e.g. your first Codex bill and what you thought was causing it vs what actually was

Vulnerability Moment

[DANIEL TO FILL] -- e.g. running what you thought was one quick task and watching the reasoning chain spin for 4 minutes on a problem you could have scoped in 30 seconds